

Oracle SPARC Solaris

- Esempio di SO a 64-bit con HW strettamente integrato
 - Obiettivi: efficienza e basso overhead
- Basato su hashing, ma più complesso
- Utilizza due hash table
 - Una per il **kernel** ed una per i **processi utente**
 - Ognuna mappa indirizzi da memoria virtuale su memoria fisica
 - Ogni entry rappresenta un'area contigua di memoria virtuale mappata
 - ▶ Non una entry di hash-table separata per ogni pagina
 - ▶ Ogni entry ha un indirizzo base e uno span (che indica il numero di pagine che la entry rappresenta)

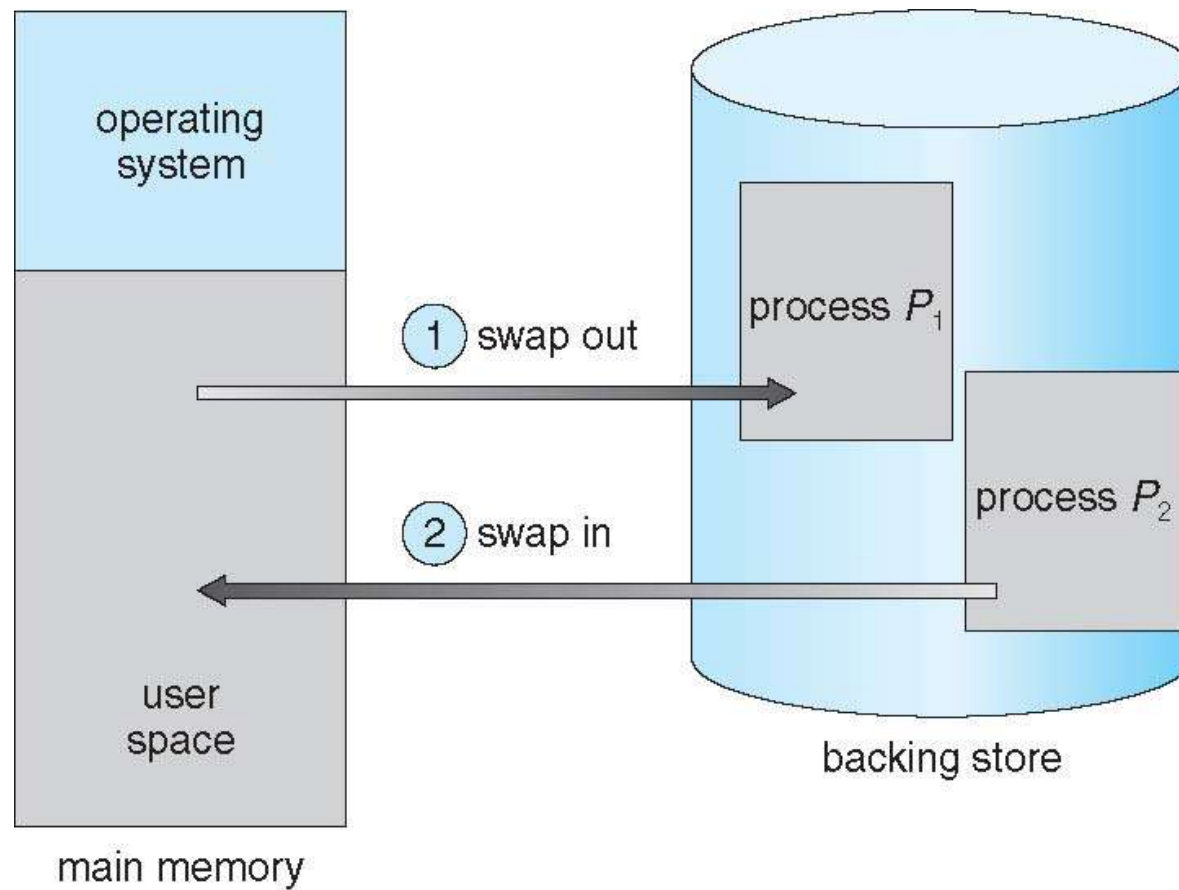
Oracle SPARC Solaris

- La ricerca di una traduzione direttamente su hash-table inefficiente
 - Si usa Translation Lookaside Buffer (TLB)
- TLB mantiene le Translation Table Entries (TTEs) per una ricerca rapida:
 - Un insieme di TTEs è anche mantenuto in un Translation Storage Buffer (TSB)
 - ▶ Include entry per le pagine con accesso recente
- Un riferimento ad un indirizzo virtuale porta alla ricerca in TLB
 - Se c'è il miss, si scandisce via hardware la TSB cercando la TTE corrispondente a quell'indirizzo (TSB walk)
 - ▶ Se si trova il match la CPU copia la TSB entry nella TLB e si completa la traduzione
 - ▶ Se non si trova il match il kernel interrompe per cercare nella hash table
 - Il kernel poi crea una TTE dalla hash table e la mette in TSB, l'interrupt handler ritorna il controllo alla MMU che completa la traduzione dell'indirizzo

Swapping

- Un processo può essere scambiato (**swapped**) temporaneamente con memoria ausiliaria (backing store) e poi portato ancora nella memoria principale per continuare l'esecuzione
 - La memoria fisica totale occupata dai processi può eccedere la memoria fisica disponibile
- **Backing store** – memoria secondaria veloce grande abbastanza per accomodare i processi che devono essere stoccati e ripristinati
 - Deve fornire un accesso diretto a queste immagini in memoria
- **Swap out, swap in** – swapping di processi
 - Processi inattivi possono essere selezionati per swapping
 - La maggior parte del tempo di swap è di tempo di trasferimento; il tempo di trasferimento è direttamente proporzionale alla quantità di memoria trasferita (swapped)

Schema dello Swapping

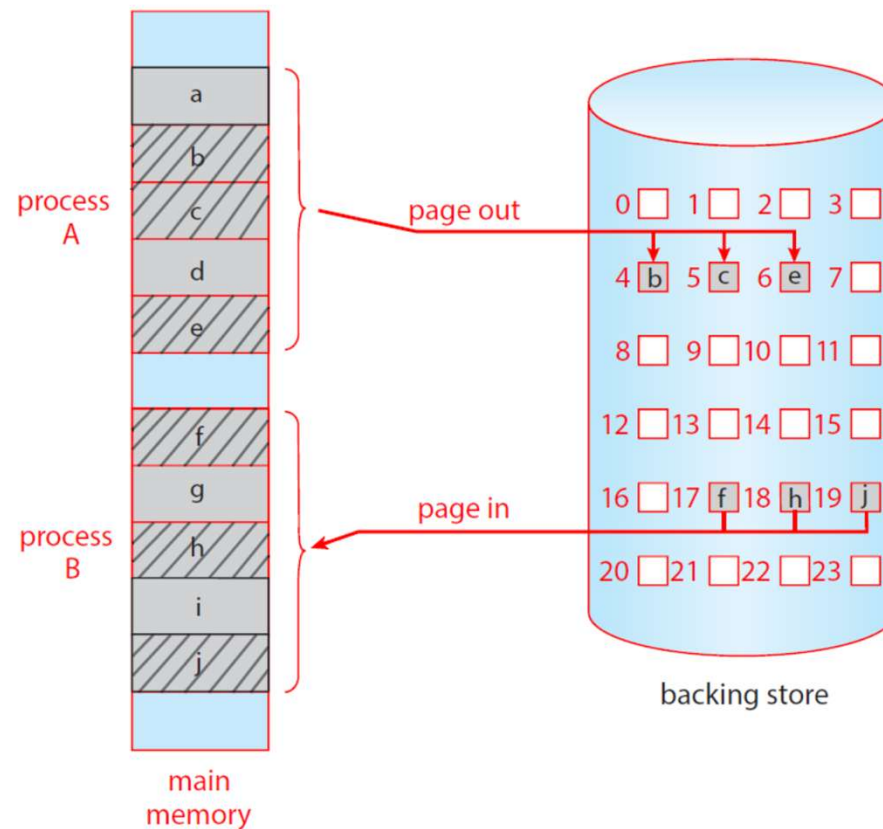


Tempo di Context Switch incluso lo Swapping

- Se il prossimo processo da mandare alla CPU non è in memoria si deve fare lo swap-out di un processo e lo swap-in del processo target
- Il tempo di context switch può essere molto alto
 - Un processo di 100MB che fa swapping su hard disk con un transfer rate di 50MB/sec
 - tempo di swap out di 2000 ms
 - ... più swap in di un processo della stessa dimensione
 - tempo totale di context switch swapping è 4000ms (4 secondi)
- Si può ridurre se si riduce la dimensione della memoria swapped
 - Sapendo quanta memoria viene usata esattamente da un processo si può ridurre

Tempo di Context Switch incluso lo Swapping

- Lo standard swapping non è più usato nei moderni SO (si usava in UNIX)
 - Una versione modificata tipica
 - ▶ Swap solo quando la memoria libera è estremamente bassa
 - Viene fatto swapping con paging
 - ▶ Sottoinsieme di pagine: page-in page-out



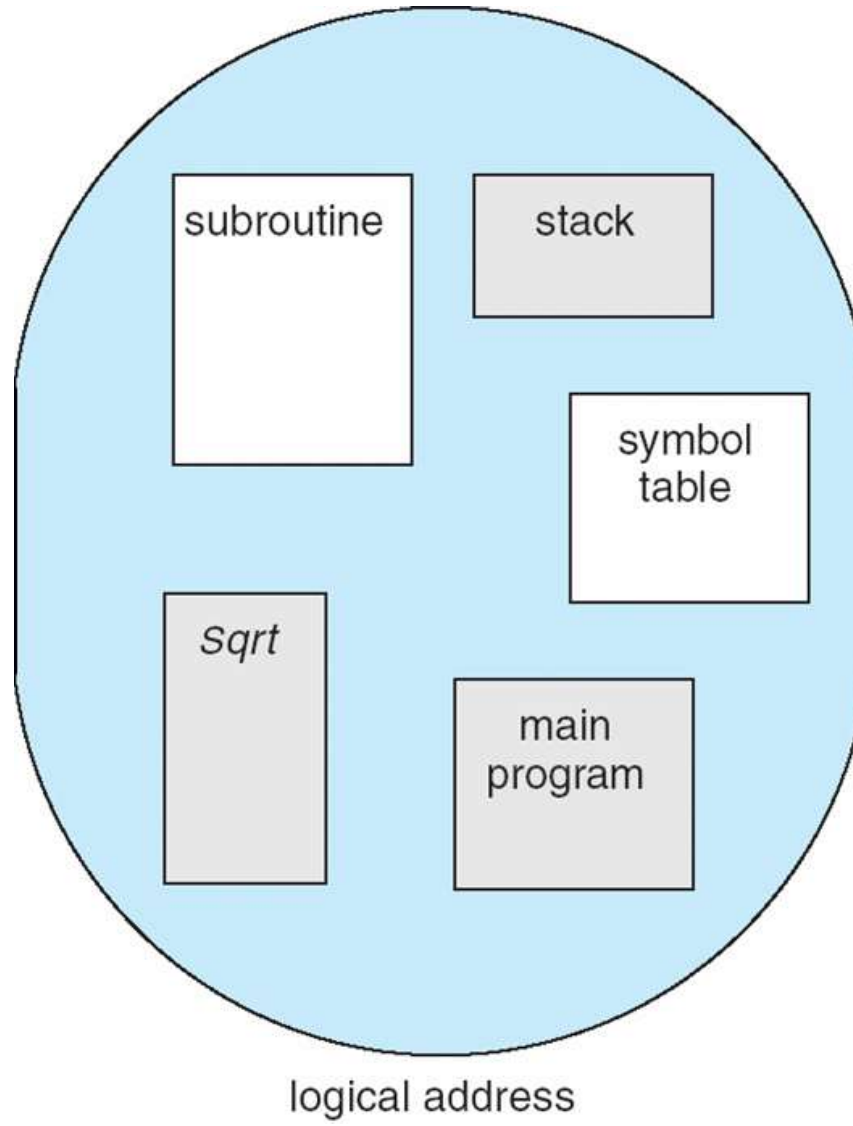
Swapping su Sistemi Mobile

- ❑ Tipicamente non supportata
 - ❑ Basta su flash memory
 - ▶ poco spazio
 - ▶ limitato numero of di cicli di scrittura
 - ▶ dialogo carente tra la flash memory e la CPU su piattaforma mobile
- ❑ Si usano altri metodi per liberare memoria
 - ❑ iOS chiede alle app di lasciare volontariamente memoria
 - ▶ Dati read-only rilasciati e ricaricati dalla flash se necessario
 - ▶ Fallimento nel rilascio può portare alla terminazione
 - ❑ Android termina le app se poca memoria, prima salva lo stato sulla flash per un restart veloce
 - ❑ Entrambi i sistemi supportano il paging

Segmentazione

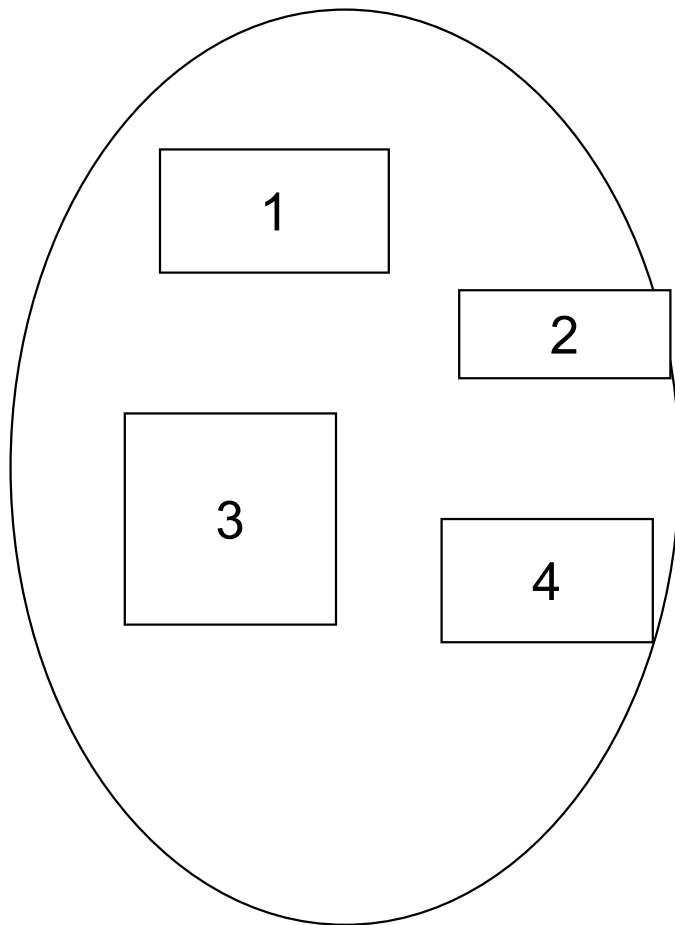
- La paginazione separa memoria logica e memoria fisica
- Segmentazione è uno schema di gestione della memoria che supporta la prospettiva utente sulla memoria
- Un programma è visto come una collezione di segmenti
 - Un segmento è un'unità logica come:
 - main program
 - procedure
 - function
 - method
 - object
 - local variables, global variables
 - common block
 - stack
 - symbol table
 - arrays

Visione utente di un Programma

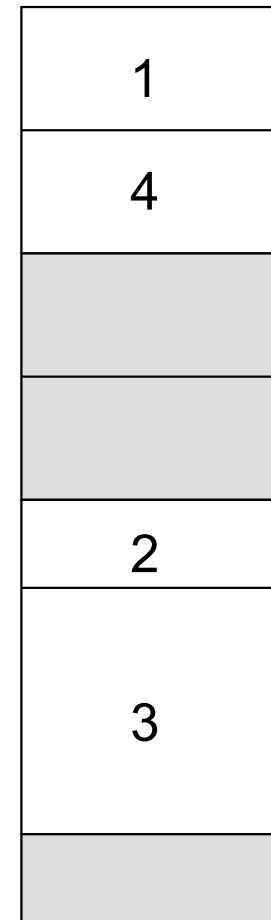


Visione Logica della Segmentazione

I segmenti sono numerati



user space

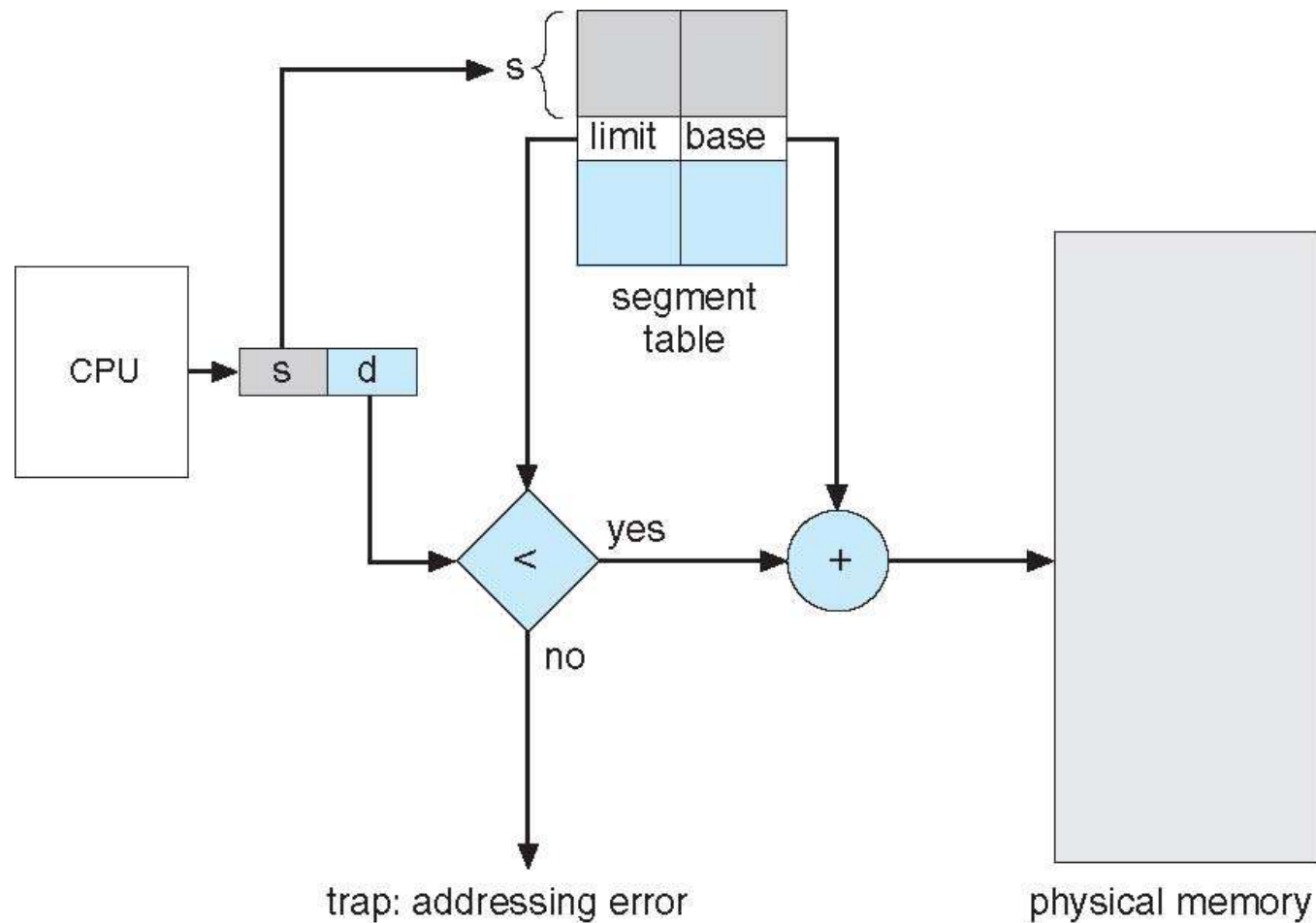


physical memory space

Architettura di Segmentazione

- Indirizzi logici sono definiti dalla tuple bidimensionale:
 <segment-number, offset>,
- Serve una struttura che mappa le tuple in indirizzi fisici
- **Tabella dei Segmenti** – mappa indirizzi bidimensionali in fisici; ogni table entry ha:
 - **base** – contiene l'indirizzo fisico di partenza del segmento
 - **limite** – specifica la lunghezza del segmento

Hardware per Segmentazione

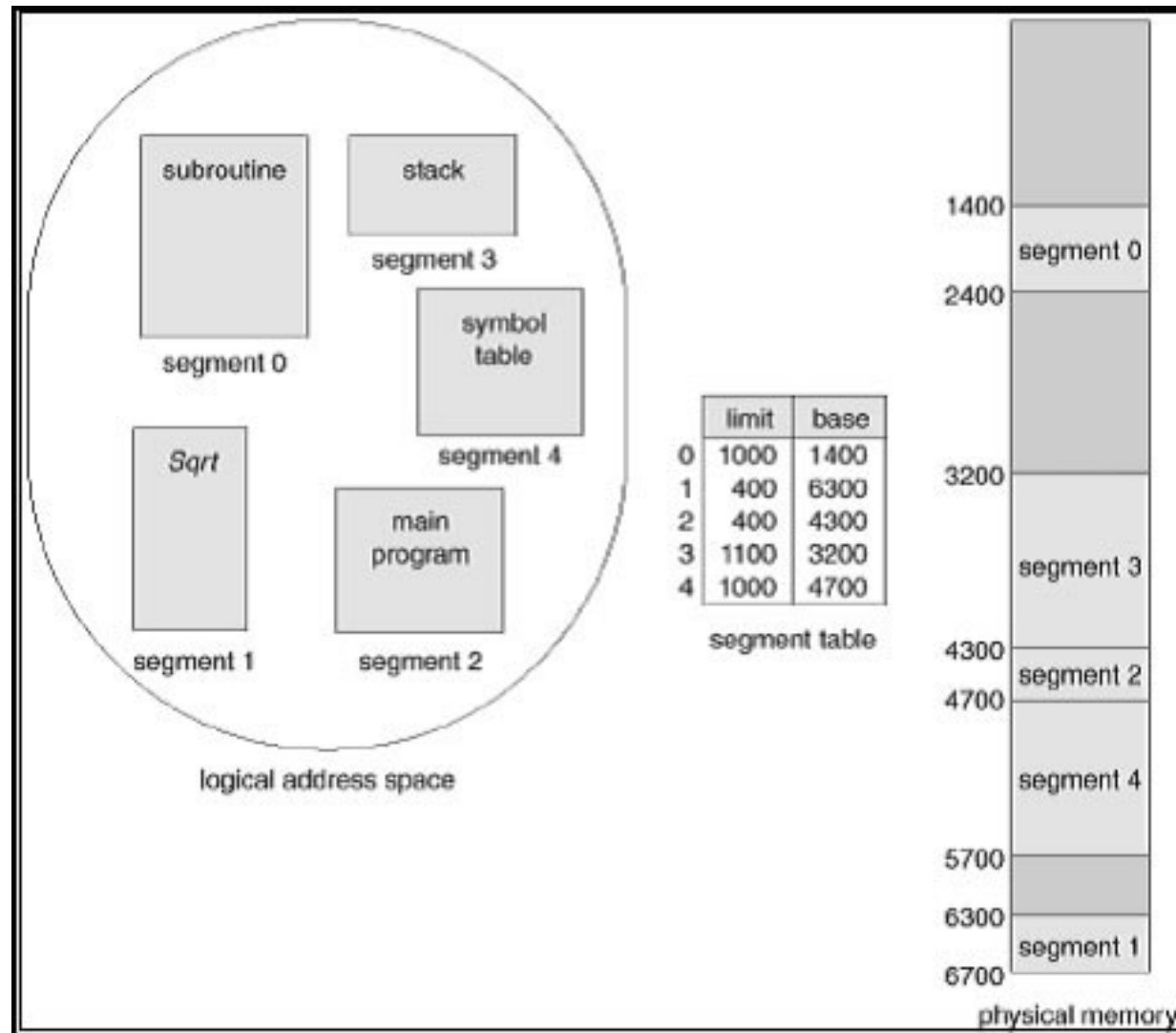


Architettura di Segmentazione

- Indirizzi logici sono definiti dalla tuple bidimensionale:
 <segment-number, offset>,
- Serve una struttura che mappa le tuple in indirizzi fisici
- **Tabella dei Segmenti** – mappa indirizzi bidimensionali in fisici; ogni table entry ha:
 - **base** – contiene l'indirizzo fisico di partenza del segmento
 - **limite** – specifica la lunghezza del segmento
- La tabella dei segmenti non può essere tenuta in registri, quindi in memoria
 - **Segment-table base register (STBR)** indica la locazione di memoria della tabella dei segmenti
 - **Segment-table length register (STLR)** indica il numero dei segmenti usati da un program:
 - ▶ Dato indirizzo logico (s,d), il numero s è legale se $s < \text{STLR}$
 - Registri associativi per limitare i due accessi in memoria

Architettura di Segmentazione

□ Esempio



Architettura per Segmentazione

- Protezione (stesso segmento, stessa protezione)
 - Ogni entry nella tabella dei segmenti è associata a:
 - ▶ validation bit = 0 $\Rightarrow$ segmento illegale
 - ▶ privilegi read/write/execute
 - Bit di protezione associati ai segmenti

- Possono essere condivisi i segmenti tra processi
 - La condivisione del codice avviene al livello di segmento
 - Come per il paging una copia di un programma condiviso (es. editor di testo)
 - Più programmi possono dover fare riferimento allo stesso numero di frammento
 - ▶ Porzioni di codice con riferimento diretto a sé stesse

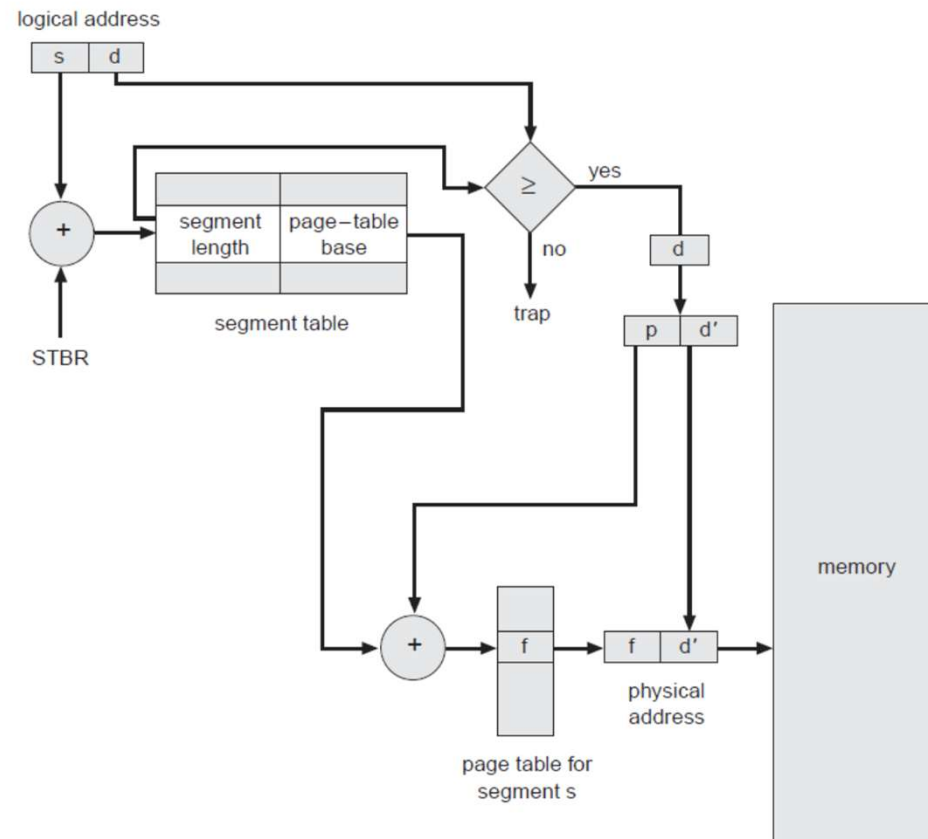
- Problema del doppio accesso come per paginazione

- I segmenti di un processo, come per pagine, non necessariamente contigui
 - segmenti di dimensione diversa quindi problemi di frammentazione
 - si possono fondere i metodi di paginazione e segmentazione

Segmentazione Paginata

□ Segmentazione paginate in GE 645 (MULTICS)

- Tabella dei segmenti:
 - ▶ lunghezza segmento e base della tabella
- Tabella delle pagine per il segment s
 - ▶ $\text{base tab} + p = \text{base pagina}$



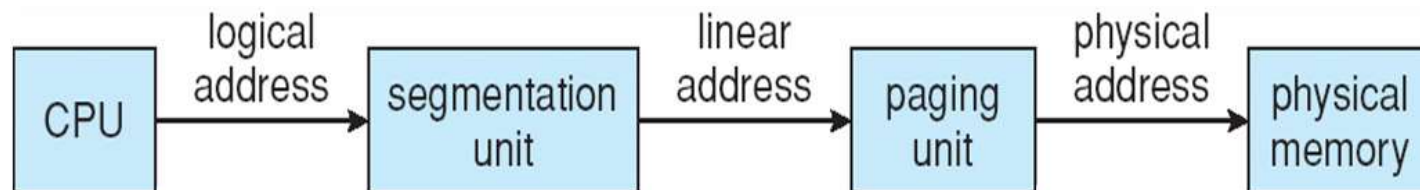
Esempi: Architetture Intel 32 e 64-bit

- Intel ha dominato l'industria per decenni
 - Negli anni 70, 8086 a 16-bit seguita da 8088 (usato nel PC IBM)
 - Poi passata ad architetture a 32-bit (Architetture IA-32 – Intel Architecture)
 - Le CPU Pentium sono esempi di 32-bit
 - Ora le CPU Intel sono a 64-bit e sono chiamate architetture IA-64

- Molte varianti nei chip, si consideriamo solo i concetti principali

Esempio: l'Architettura Intel IA-32

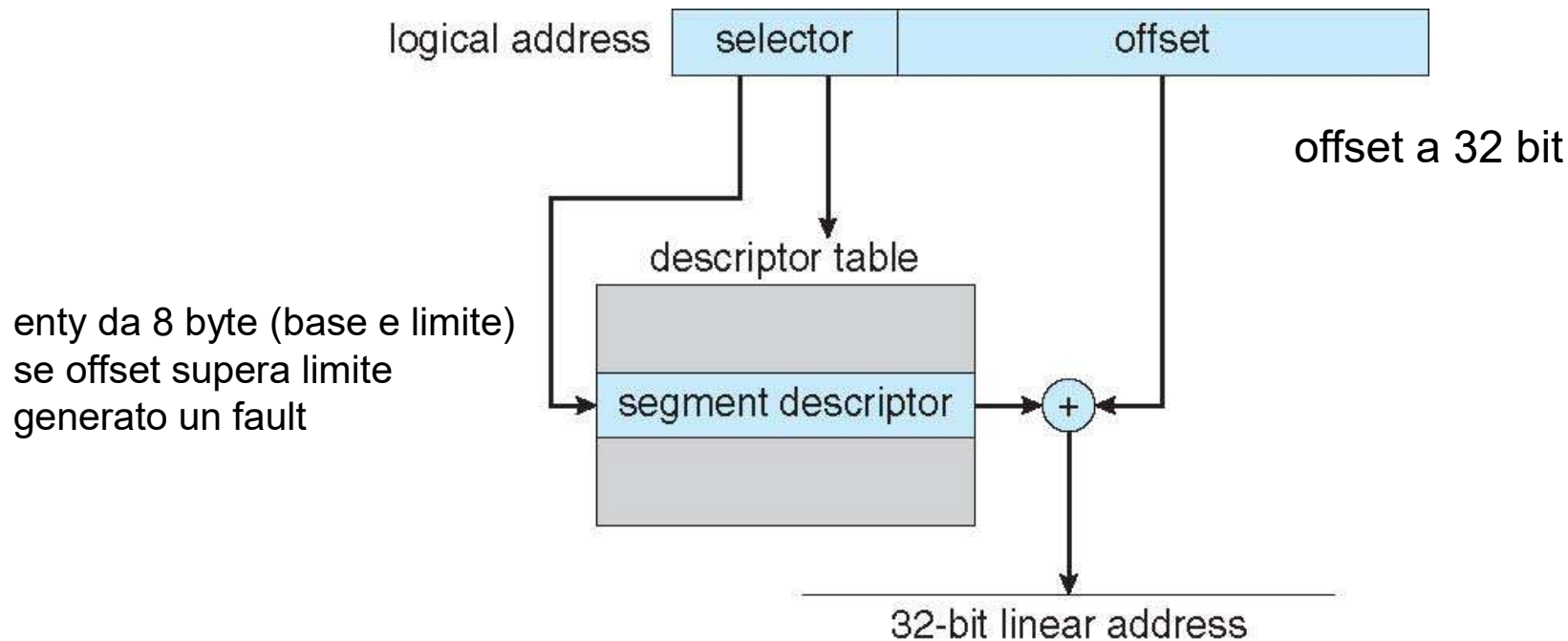
- Supporta sia segmentazione che segmentazione con paging
- La CPU genera indirizzi logici passati all'unità di segmentazione che produce **indirizzi lineari** poi passati all'unità di paging che produce l'indirizzo fisico (segmentation unit + page unit = MMU)



- Ogni segmento grande fino a 4 GB
- Numero di segmenti per processo divisi in due partizioni
 - ▶ Prima partizione segmenti privati per il processo (descritti in una **local descriptor table (LDT)**)
 - ▶ Seconda partizione segmenti condivisi tra tutti i processi (descritti in una **global descriptor table (GDT)**)

Esempio: Architettura Intel IA-32

- La CPU genera indirizzi logici per l'unità di segmentazione



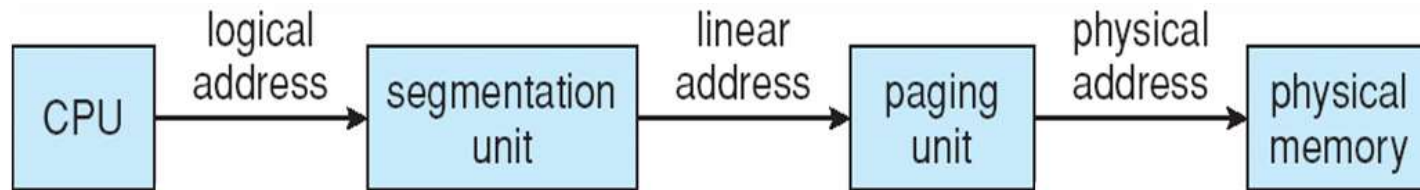
- Il selettore è a 16-bit



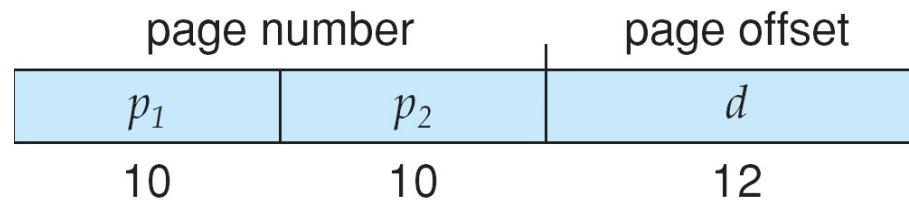
- s per il numero di segmento, g il tipo (global/local), p la protezione

Esempio: Architettura Intel IA-32

- Traduzione da indirizzo logico a fisico in IA-32

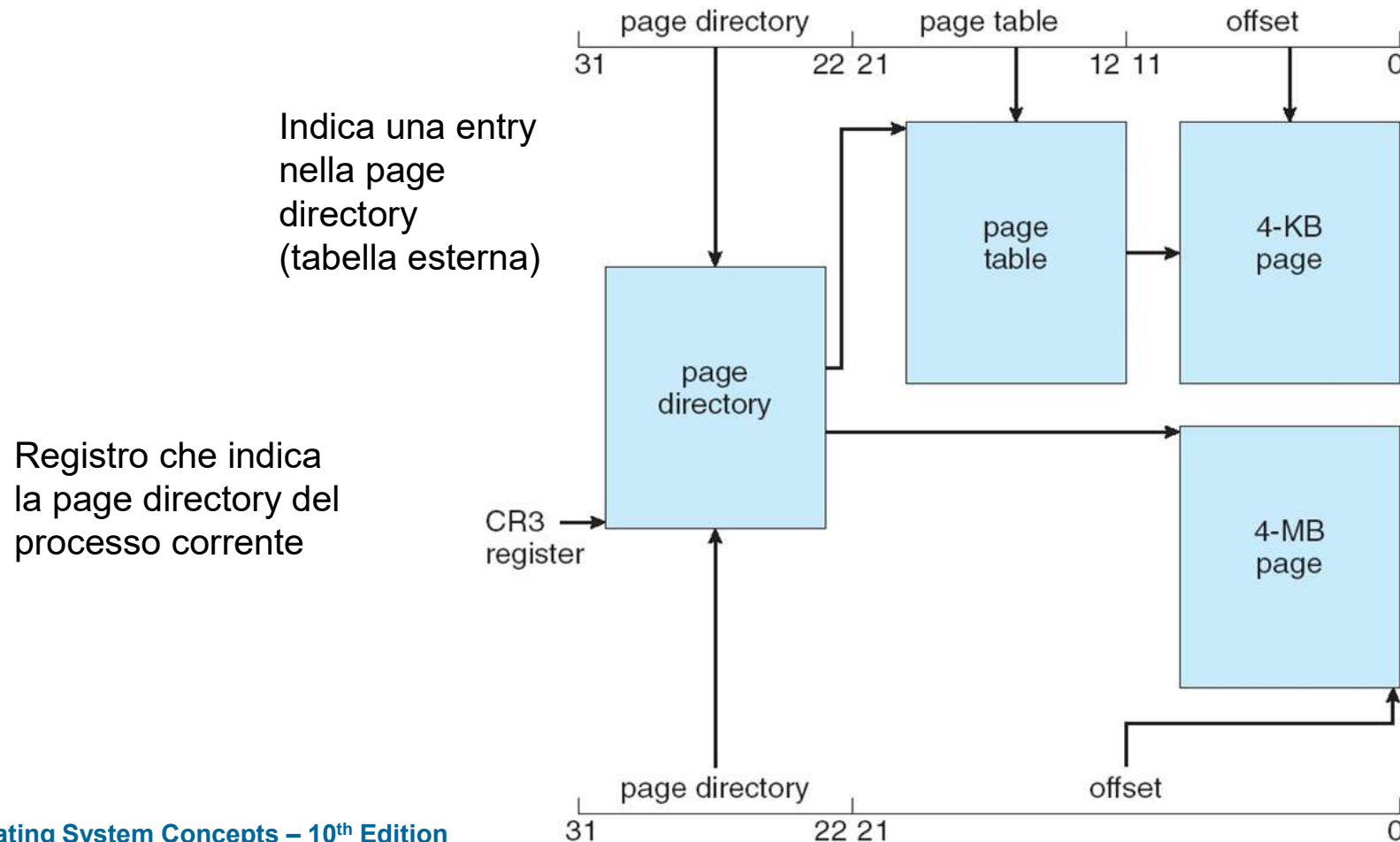


- Unità di paging di IA-32
- Dimensioni delle pagine 4 KB o 4 MB
 - Per 4 KB schema di paging a due livelli
 - Per 4 MB schema ad un livello
 - Indirizzo lineare 32 bit per paging a 2 livelli



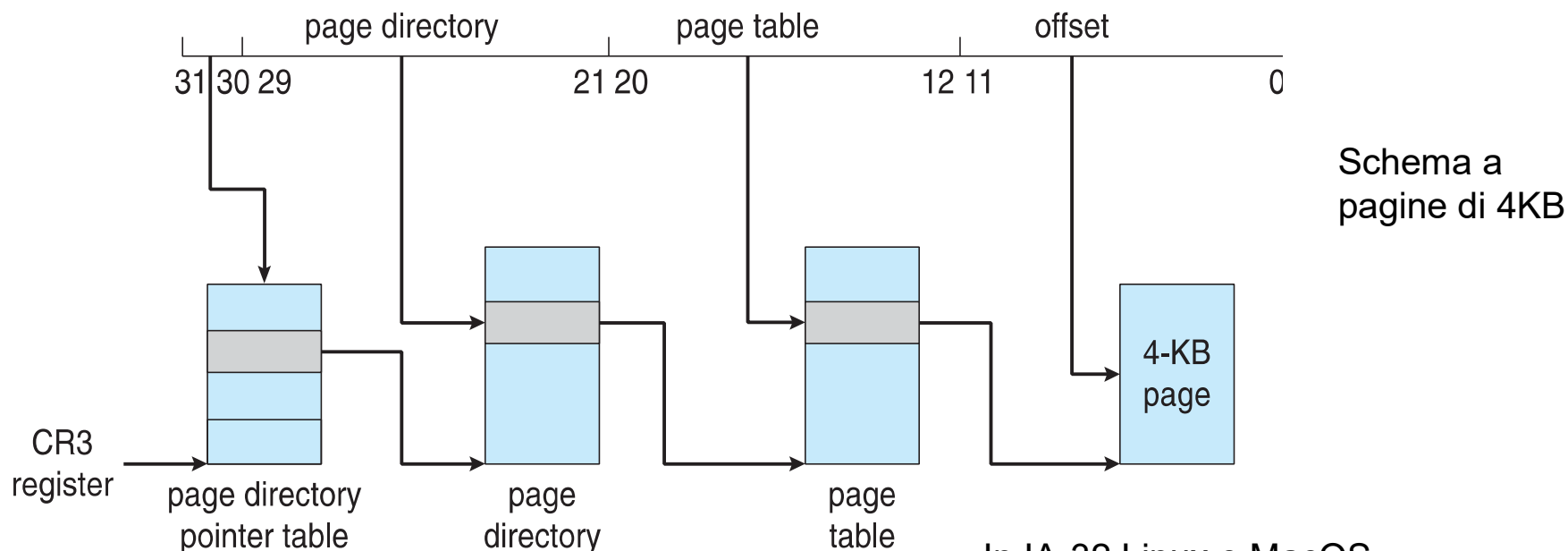
Esempio: Architettura Intel IA-32

- Unità di paging di IA-32
- Dimensioni delle pagine 4 KB o 4 MB
 - Per 4 KB schema di paging a due livelli
 - Per 4 MB schema ad un livello



Intel IA-32 Estensioni Indirizzo Pagina

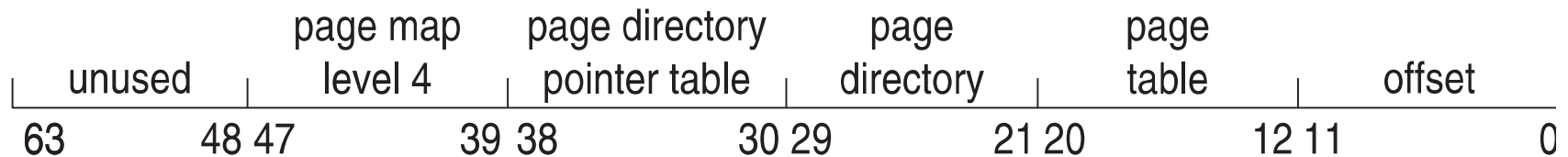
- Limiti di accesso di indirizzi 32-bit hanno portato Intel a creare **page address extension (PAE)** permettendo ad app di 32-bit l'accesso a memorie più grandi di 4GB
 - Paging con schema a 3-livelli (per pagine a 4 KB)
 - Primi 2 bit si riferiscono ad una **page directory pointer table**
 - Le entry di page-directory e page-table aumentate da 32 a 64-bit
 - ▶ Base address da 20 a 24 bit + 12 bit per offset
 - ▶ Indirizzi fisici di 36 bit – fino a 64GB di memoria fisica



In IA-32 Linux e MacOS supportano, Windows no

Intel x86-64

- Generazione corrente è architettura x86-64
- x86-64 proposto da AMD e adottato da Intel (AMD64 e Intel64)
- 64 bit consentirebbero dimensioni enormi ($2^{64} > 16$ exabyte)
- In pratica è implementato un indirizzamento a 48 bit
 - Dimensione di pagina 4 KB, 2 MB, 1 GB
 - Quattro livelli di gerarchia di paging



Anche se 48-bit virtuali con page address extension fino a 52-bit indirizzi fisici

Esempio: Architettura ARM

- Dominante per i chip su piattaforma mobile (es. Apple iOS e Google Android, ma anche sistemi embedded real-time)
- CPU moderna ed efficiente energeticamente
- ARMv8 è a 64-bit

Esempio: Architettura ARMv8

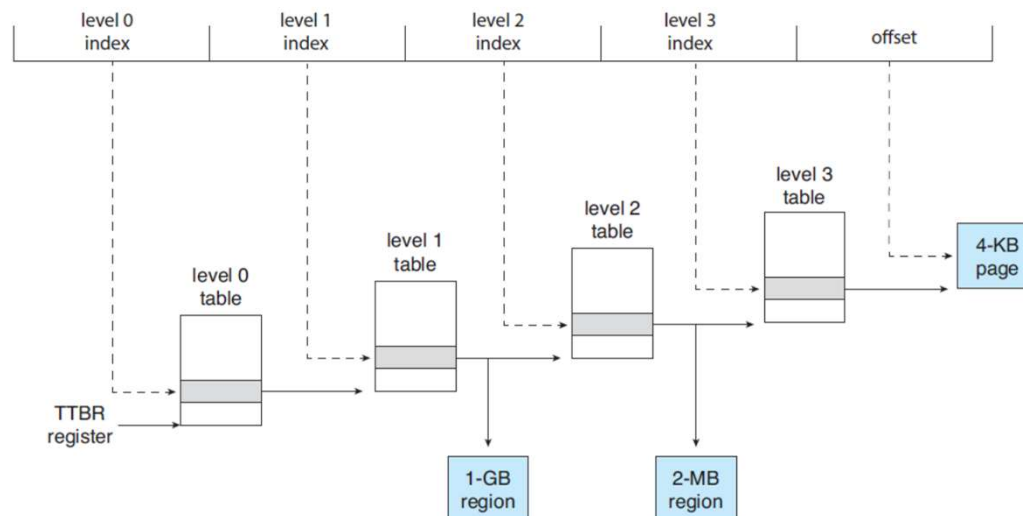
- ARMv8 è a 64-bit
- Tre granularità di traduzione 4 KB, 16 KB, 64 KB associate a dimensione di pagina e sezioni di memoria contigua (regions)

Translation Granule Size	Page Size	Region Size
4 KB	4 KB	2 MB, 1 GB
16 KB	16 KB	32 MB
64 KB	64 KB	512 MB

- Usati fino a 48 bit



- Per 4 KB e 16 KB fino a 4 livelli di paging, fino a 3 per 64 KB



Esempio: Architettura ARMv8

- ARMv8 è a 64-bit
- Tre granularità di traduzione 4 KB, 16 KB, 64 KB associate a dimensione di pagina e sezioni di memoria contigua (regions)

Translation Granule Size	Page Size	Region Size
4 KB	4 KB	2 MB, 1 GB
16 KB	16 KB	32 MB
64 KB	64 KB	512 MB

- Usati fino a 48 bit



- Per 4 KB e 16 KB fino a 4 livelli di paging, fino a 3 per 64 KB
- I livelli 1 e 2 possono essere anche usati per indirizzare regioni da 1-GB (livello 1) e 2-MB (livello 2)
 - Se livello 1, 0–30 bit di offset, se livello 2, 0–20 bit di offset
 - Due livelli di TLBs
 - ▶ Livelli interni hanno due micro TLBs (una dati, una istruzioni)
 - ▶ La più esterna è una singola TLB

Comandi

- Layout di memoria di un processo
 - pmap [options] [pid]
 - ▶ -x extended info (address, permissions)
 - ▶ -d content of heap regions
 - ▶ -p show the page size
 - pmap -d [pid]
 - pmap -x [pid]
 - ps

Address	Kbytes	RSS	Dirty	Mode	Mapping
0000563511c54000	2048	1024	0	r-x--	/path/to/executable
0000563511e54000	4	4	4	rw---	/path/to/executable
00007f5c0ec75000	16384	8192	0	rw---	[anon]